

SMART FILE AUTOMATION SYSTEM

Task 1: Python File Handling & Automation

Submitted By:

Ishita Gupta

Organization:

Alfido Tech Internship Program

Technology:

Python

Date:

June 2026

ABSTRACT

The Smart File Automation System is a Python-based application developed to automate file management operations. The project demonstrates file handling, CSV processing, activity logging, exception handling, and modular programming concepts.

OBJECTIVES

- Understand Python file handling.
- Implement file automation operations.
- Work with CSV files.
- Apply exception handling.
- Maintain activity logs.

FEATURES IMPLEMENTED

1. Create File
2. Read File
3. Write File
4. Rename File
5. Move File

6. Delete File

7. Read CSV File

8. Activity Logging

TECHNOLOGIES USED

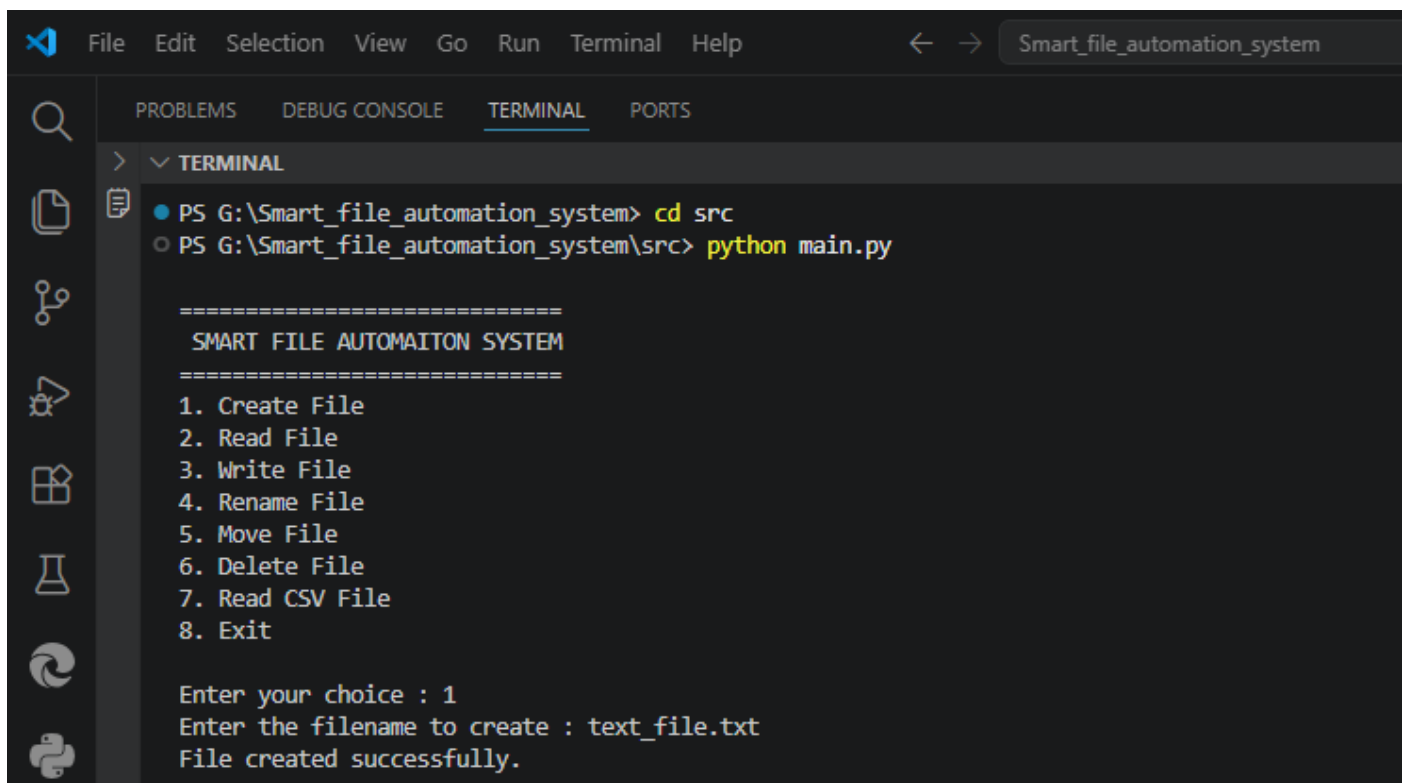
- Python
- OS Module
- SHUTIL Module
- CSV Module
- Datetime Module
- Visual Studio Code

PROJECT STRUCTURE

Smart File Automation System

```
|
|
├── src
├── data
├── logs
├── output
├── screenshots
├── docs
├── README.md
└── requirements.txt
```

Figure 1: File Creation Operation



The screenshot shows the Visual Studio Code interface with the 'Smart_file_automation_system' project open. The 'TERMINAL' tab is active, displaying the following commands and output:

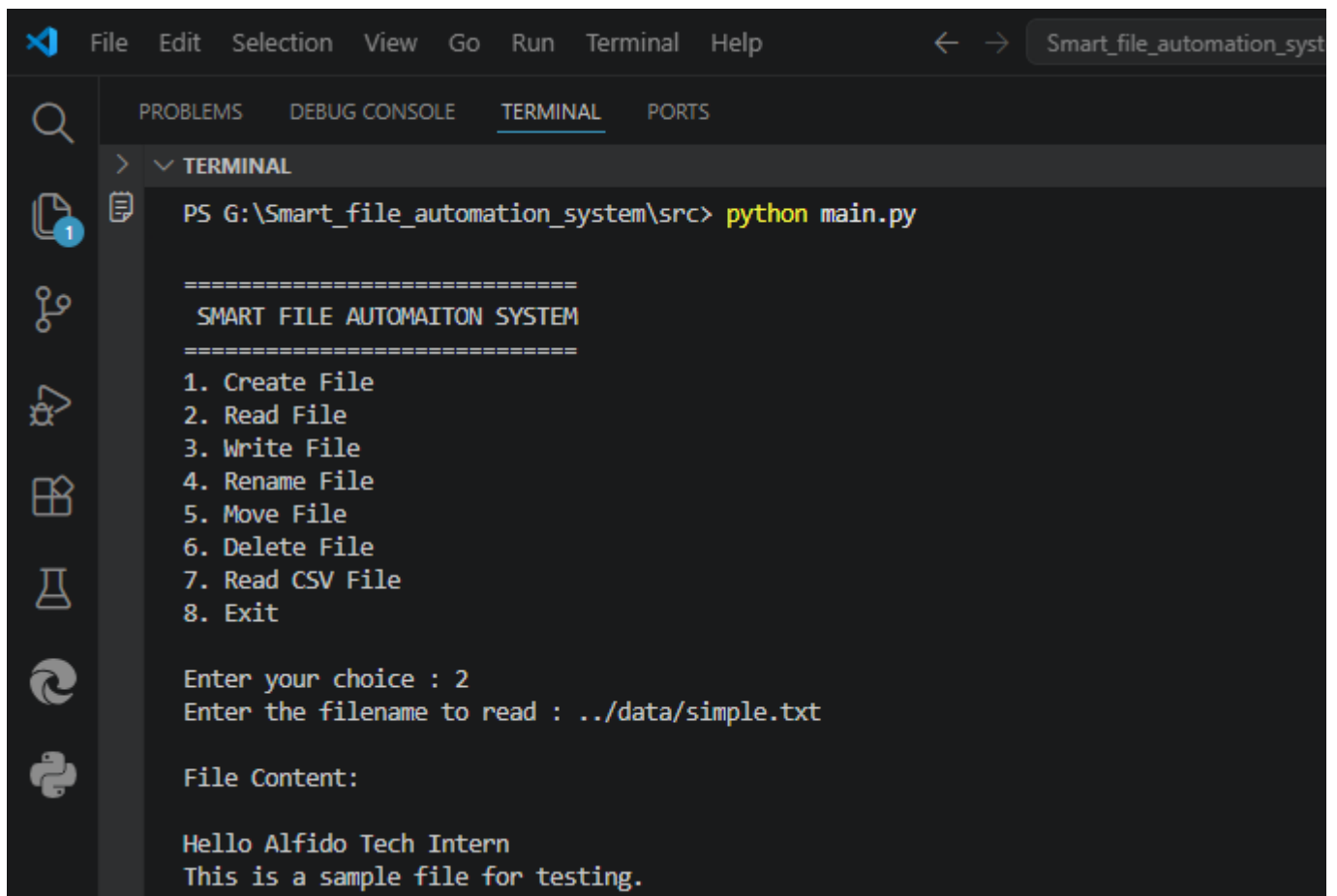
```
PS G:\Smart_file_automation_system> cd src
PS G:\Smart_file_automation_system\src> python main.py

=====
SMART FILE AUTOMAITON SYSTEM
=====

1. Create File
2. Read File
3. Write File
4. Rename File
5. Move File
6. Delete File
7. Read CSV File
8. Exit

Enter your choice : 1
Enter the filename to create : text_file.txt
File created successfully.
```

Figure 2: File Reading operation



The screenshot shows the Visual Studio Code interface with the 'Smart_file_automation_system' project open. The 'TERMINAL' tab is active, displaying the following commands and output:

```
PS G:\Smart_file_automation_system\src> python main.py

=====
SMART FILE AUTOMAITON SYSTEM
=====

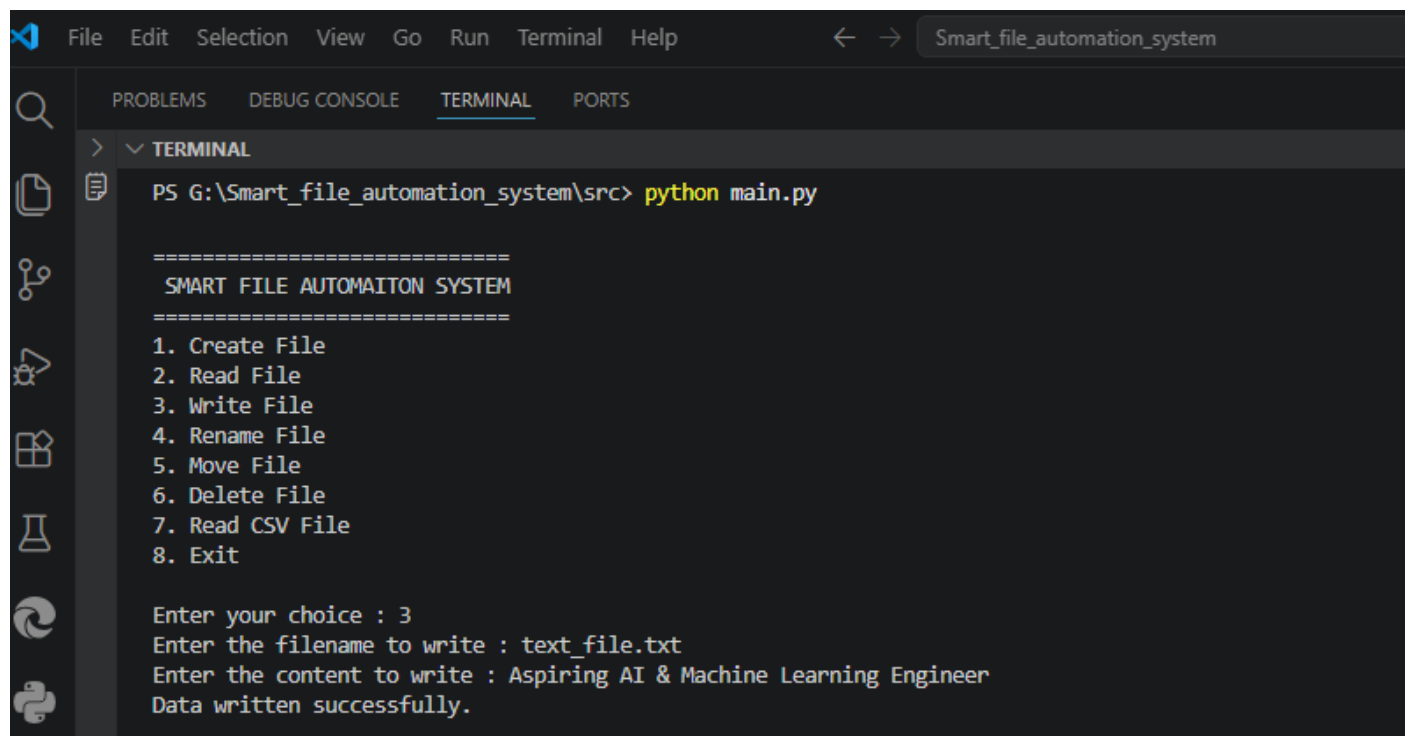
1. Create File
2. Read File
3. Write File
4. Rename File
5. Move File
6. Delete File
7. Read CSV File
8. Exit

Enter your choice : 2
Enter the filename to read : ../data/simple.txt

File Content:

Hello Alfido Tech Intern
This is a sample file for testing.
```

Figure 3: File Writing Operation

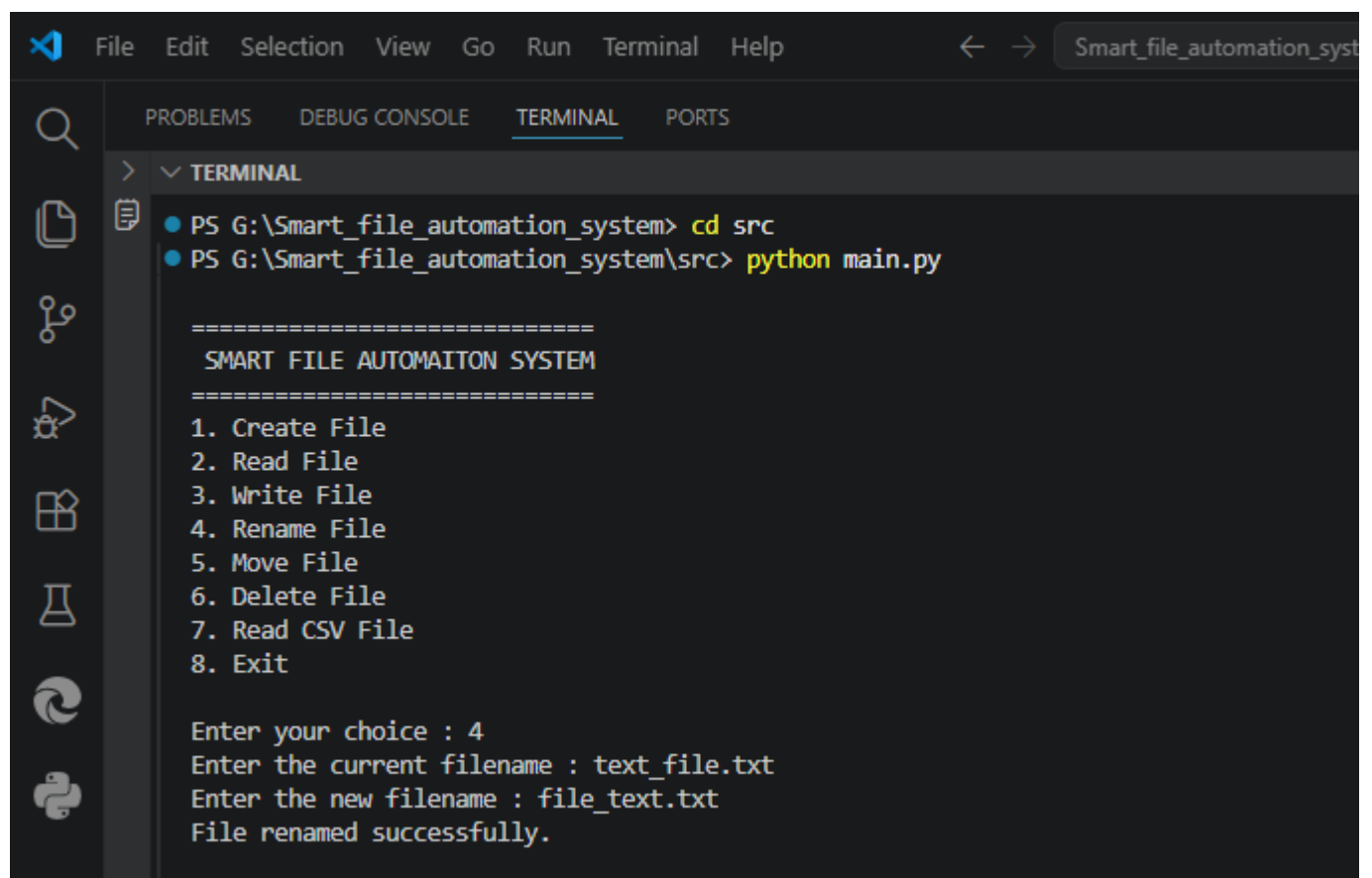
A screenshot of the Visual Studio Code interface. The top menu bar includes File, Edit, Selection, View, Go, Run, Terminal, and Help. The breadcrumb at the top right shows the path 'Smart_file_automation_system'. The left sidebar contains icons for Explorer, Search, Source Control, Run and Debug, Extensions, Testing, and Python. The main area is the 'TERMINAL' tab, showing a command prompt session. The prompt is 'PS G:\Smart_file_automation_system\src> python main.py'. The output shows a menu for 'SMART FILE AUTOMAITON SYSTEM' with 8 options. Option 3, 'Write File', is selected. The user is prompted to enter a filename and content. The filename 'text_file.txt' and content 'Aspiring AI & Machine Learning Engineer' are entered. The output concludes with 'Data written successfully.'

```
PS G:\Smart_file_automation_system\src> python main.py

=====
SMART FILE AUTOMAITON SYSTEM
=====
1. Create File
2. Read File
3. Write File
4. Rename File
5. Move File
6. Delete File
7. Read CSV File
8. Exit

Enter your choice : 3
Enter the filename to write : text_file.txt
Enter the content to write : Aspiring AI & Machine Learning Engineer
Data written successfully.
```

Figure 4: File Renaming Operation

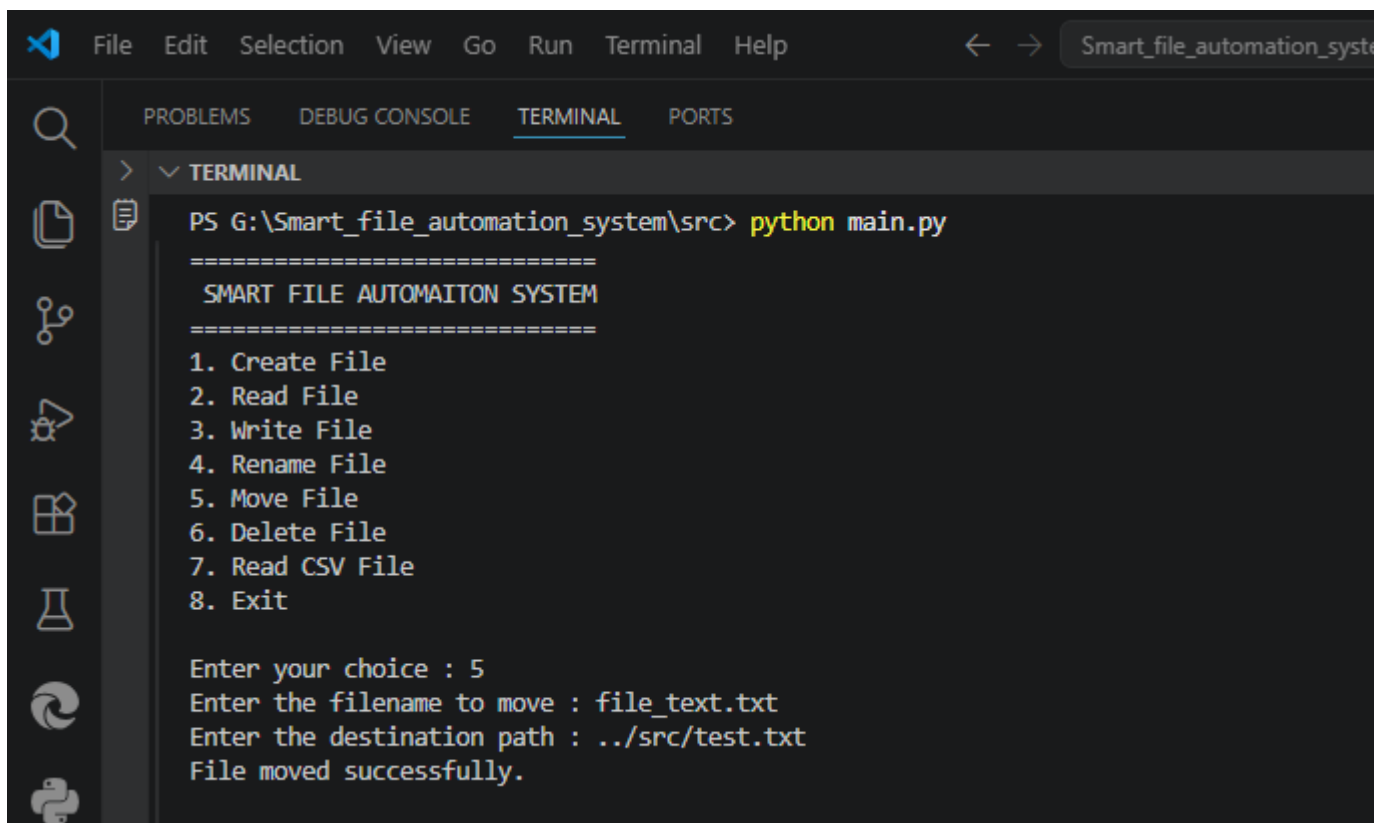
A screenshot of the Visual Studio Code interface. The top menu bar includes File, Edit, Selection, View, Go, Run, Terminal, and Help. The breadcrumb at the top right shows the path 'Smart_file_automation_syst'. The left sidebar contains icons for Explorer, Search, Source Control, Run and Debug, Extensions, Testing, and Python. The main area is the 'TERMINAL' tab, showing a command prompt session. The prompt is 'PS G:\Smart_file_automation_system> cd src'. The output shows a menu for 'SMART FILE AUTOMAITON SYSTEM' with 8 options. Option 4, 'Rename File', is selected. The user is prompted to enter the current filename and the new filename. The current filename 'text_file.txt' and new filename 'file_text.txt' are entered. The output concludes with 'File renamed successfully.'

```
PS G:\Smart_file_automation_system> cd src
PS G:\Smart_file_automation_system\src> python main.py

=====
SMART FILE AUTOMAITON SYSTEM
=====
1. Create File
2. Read File
3. Write File
4. Rename File
5. Move File
6. Delete File
7. Read CSV File
8. Exit

Enter your choice : 4
Enter the current filename : text_file.txt
Enter the new filename : file_text.txt
File renamed successfully.
```

Figure 5 File Moving Operation



The screenshot shows a Visual Studio Code interface with the 'Terminal' tab active. The terminal window displays the output of a Python script named 'main.py' executed from the directory 'G:\Smart_file_automation_system\src'. The script prompts the user to enter a choice from a list of operations. The user has entered '5', which corresponds to 'Move File'. The script then prompts for the filename to move ('file_text.txt') and the destination path ('../src/test.txt'), and finally confirms that the file was moved successfully.

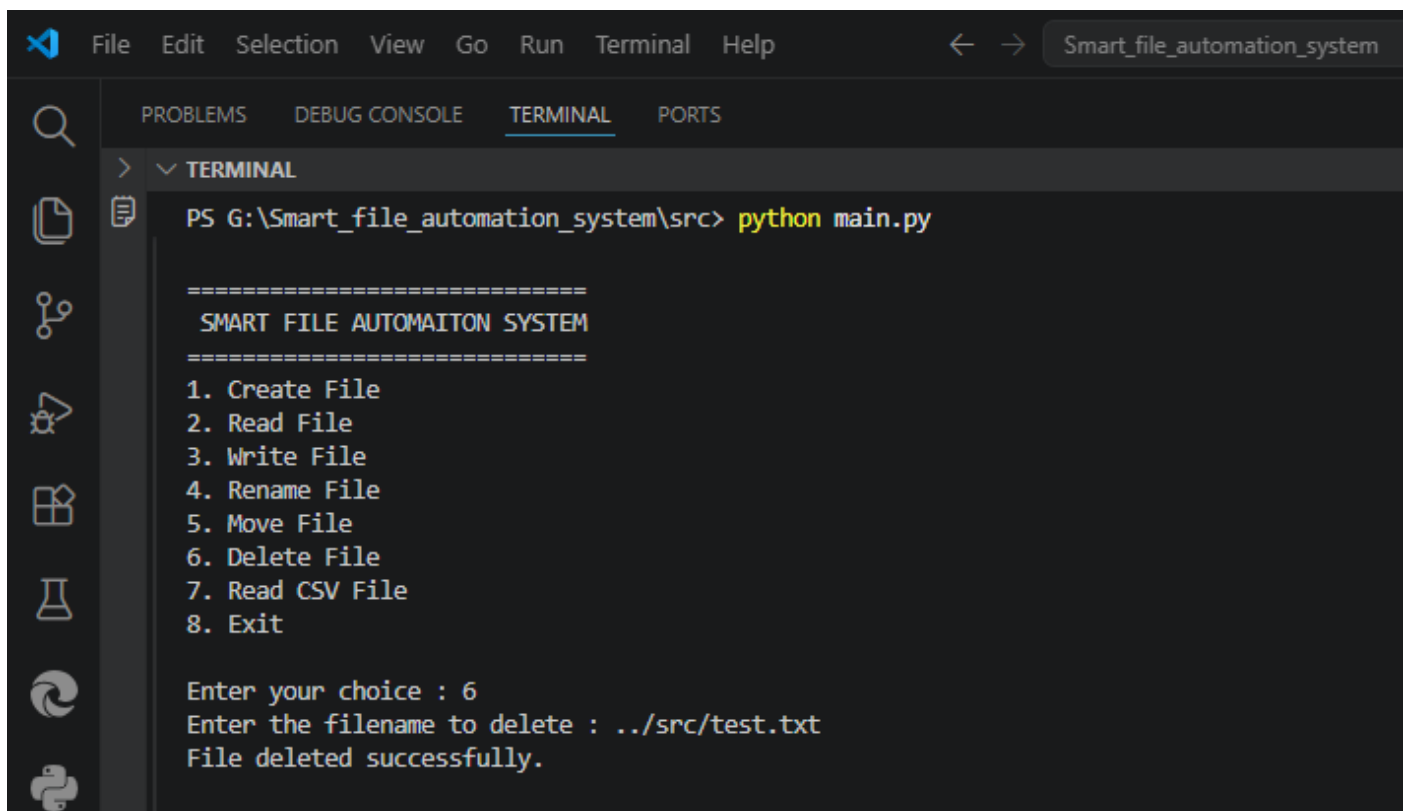
```
PS G:\Smart_file_automation_system\src> python main.py

=====
SMART FILE AUTOMAITON SYSTEM
=====

1. Create File
2. Read File
3. Write File
4. Rename File
5. Move File
6. Delete File
7. Read CSV File
8. Exit

Enter your choice : 5
Enter the filename to move : file_text.txt
Enter the destination path : ../src/test.txt
File moved successfully.
```

Figure 6: File Deleting Operation



The screenshot shows a Visual Studio Code interface with the 'Terminal' tab active. The terminal window displays the output of a Python script named 'main.py' executed from the directory 'G:\Smart_file_automation_system\src'. The script prompts the user to enter a choice from a list of operations. The user has entered '6', which corresponds to 'Delete File'. The script then prompts for the filename to delete ('../src/test.txt') and finally confirms that the file was deleted successfully.

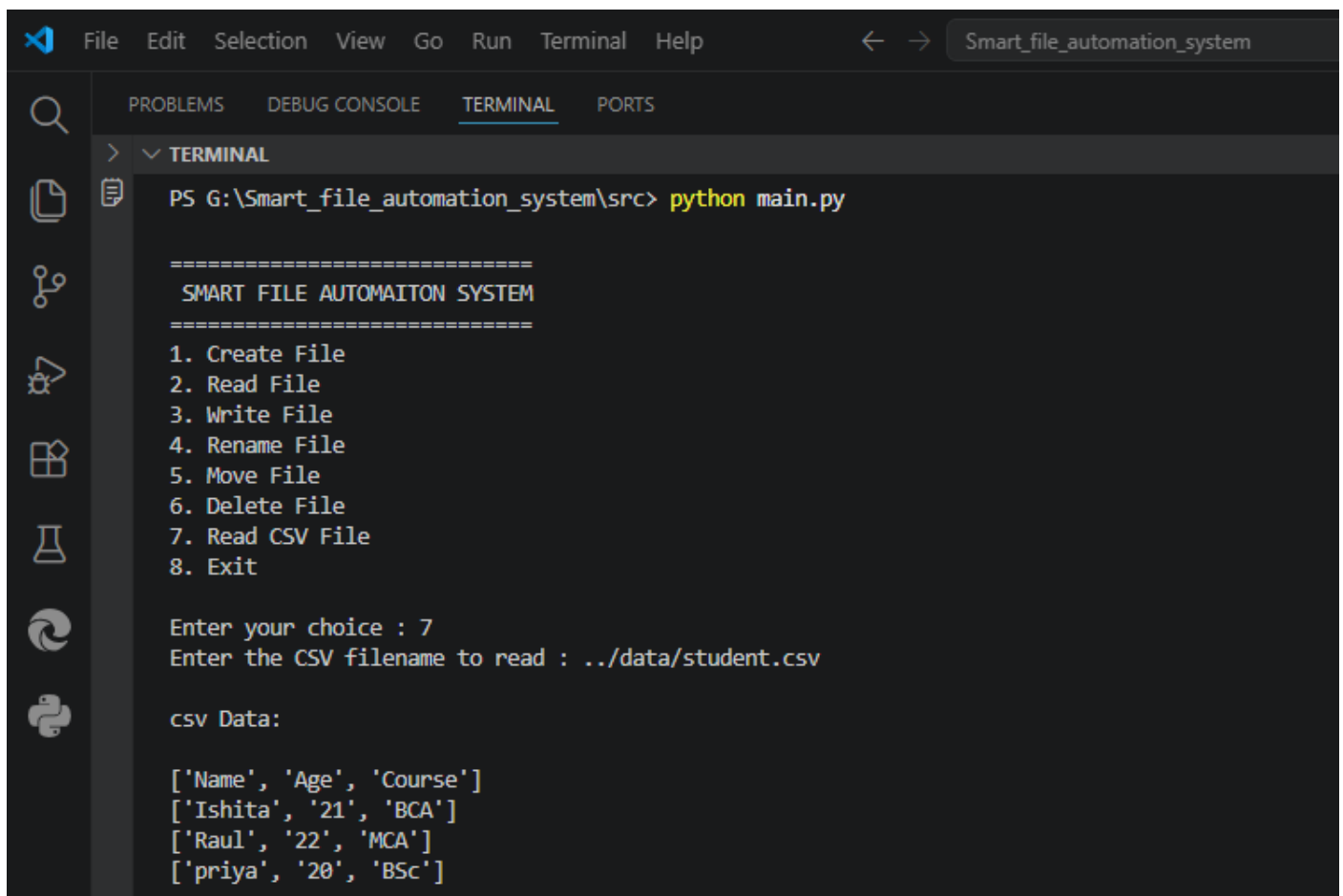
```
PS G:\Smart_file_automation_system\src> python main.py

=====
SMART FILE AUTOMAITON SYSTEM
=====

1. Create File
2. Read File
3. Write File
4. Rename File
5. Move File
6. Delete File
7. Read CSV File
8. Exit

Enter your choice : 6
Enter the filename to delete : ../src/test.txt
File deleted successfully.
```

Figure 7: Reading CSV File Operation



The screenshot shows a Visual Studio Code interface with the 'Smart_file_automation_system' project open. The 'TERMINAL' tab is active, displaying the output of a Python script. The script prompts the user to enter a choice from a menu of file operations. Choice 7, 'Read CSV File', is selected. The script then prompts for the CSV filename, and the user enters '../data/student.csv'. The script successfully reads the CSV data and displays it as a list of lists.

```
PS G:\Smart_file_automation_system\src> python main.py

=====
SMART FILE AUTOMAITON SYSTEM
=====

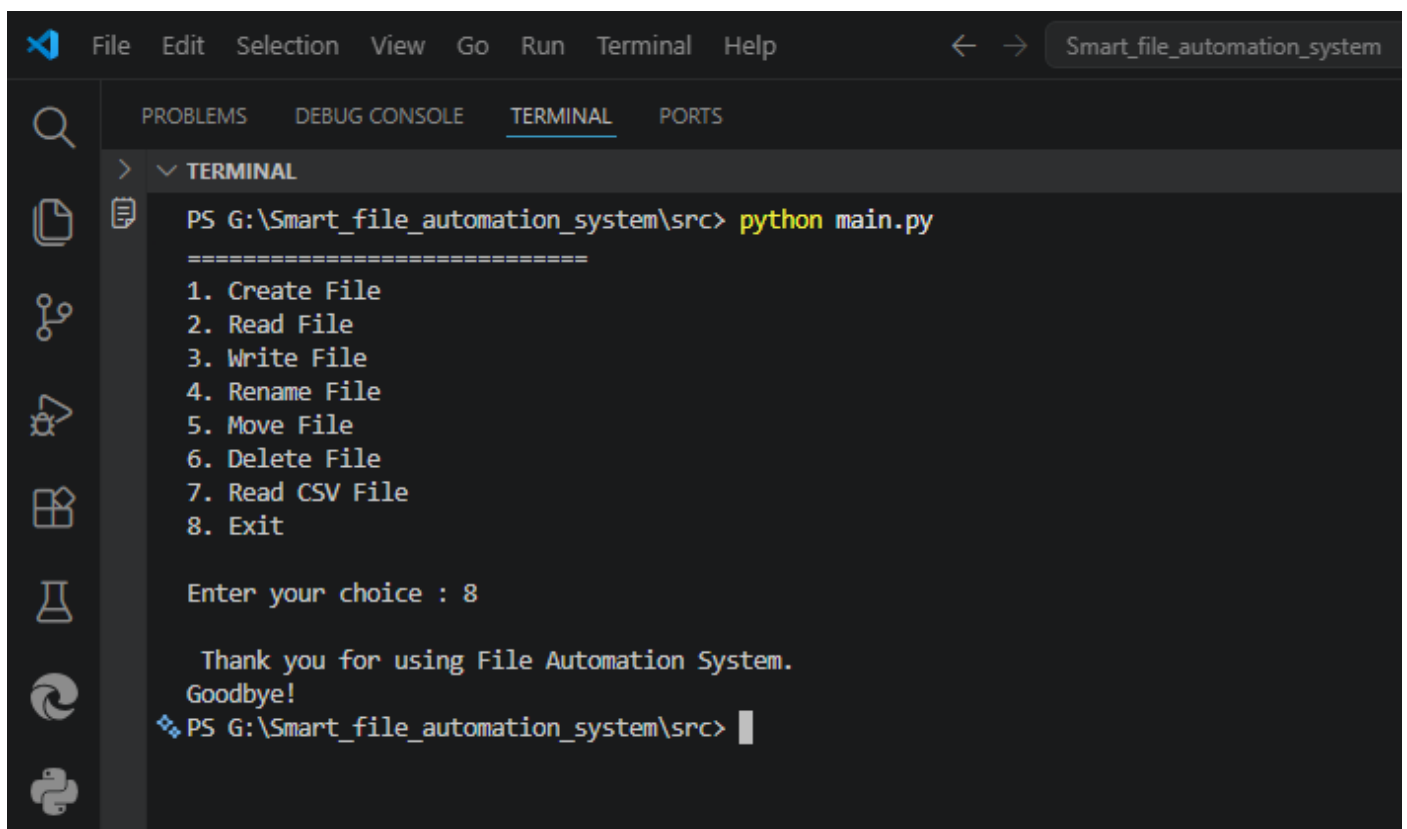
1. Create File
2. Read File
3. Write File
4. Rename File
5. Move File
6. Delete File
7. Read CSV File
8. Exit

Enter your choice : 7
Enter the CSV filename to read : ../data/student.csv

csv Data:

['Name', 'Age', 'Course']
['Ishita', '21', 'BCA']
['Raul', '22', 'MCA']
['priya', '20', 'BSc']
```

Figure 8: Exit Operation



The screenshot shows the same Visual Studio Code interface as Figure 7. The 'TERMINAL' tab is active, displaying the output of the Python script. The user has now selected choice 8, 'Exit'. The script displays a thank you message and a goodbye message before returning to the command prompt.

```
PS G:\Smart_file_automation_system\src> python main.py

=====
1. Create File
2. Read File
3. Write File
4. Rename File
5. Move File
6. Delete File
7. Read CSV File
8. Exit

Enter your choice : 8

Thank you for using File Automation System.
Goodbye!
❖ PS G:\Smart_file_automation_system\src> |
```

RESULTS

The Smart File Automation System was successfully tested for all implemented functionalities. The application correctly performed file creation, reading, writing, renaming, moving, deletion, CSV file processing, and activity logging. All operations were executed successfully with proper exception handling.

CONCLUSION

The Smart File Automation System successfully automates common file management operations using Python. The project enhanced practical understanding of file handling, modular programming, exception handling, and logging mechanisms.